

Project Report

Cache-Aware and Cache-Oblivious Algorithms for String Sorting

Kausheya Roy Harshit Lalwani

May 2026

1 Problem Statement

This project studies cache-aware and cache-oblivious algorithms with the goal of designing and implementing efficient solutions for **String Sorting**. We evaluate these algorithms based on their ability to minimise I/O operations $Q(N; M; B)$ between a fast internal memory of size M and a larger memory, where data is transferred in blocks of size B . The primary metric of success is how closely an algorithm approaches the scanning bound $\text{Scan}(N) = O(N/B)$.

Table ?? summarises the theoretical I/O and time complexity of every algorithm studied.

Table 1: Theoretical complexity of the studied algorithms. N : number of strings; $D = \sum_{i=1}^N d_i$: total distinguishing prefix length; d_i : distinguishing prefix length of string i ; L : burst threshold; B : block size; M : cache size; ℓ : mean string length.

Algorithm	Time complexity	I/O $Q(N; M; B)$	Cache model
<code>std::sort</code>	$O(N \log N \cdot \bar{\ell})$	$O\left(\frac{N}{B} \log N\right)$	Cache-agnostic
Multikey QS	$O(D + N \log N)$	$O\left(\frac{N}{B} \log N\right)$	Cache-agnostic
MSD Radix Sort	$O(D)$	$O\left(\frac{D}{B}\right)$ if $ \Sigma \leq M/B$	Cache-aware
Burtsort	$O(D + N \log L)$	$O\left(\frac{N}{B} \cdot \frac{D}{N \cdot L}\right)$	Cache-aware
C-Burtsort	$O(D + N \log L)$	$O\left(\frac{N}{B} \cdot \frac{D}{N \cdot L}\right)$	Cache-aware
Funnelsort (lazy)	$O(N \log N)$	$O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$	Cache-oblivious

2 The Architecture Problem with Baseline Algorithms

Standard algorithms often ignore the memory hierarchy, leading to significant performance degradation on large datasets.

2.1 Standard Introsort (`std::sort`)

Standard implementations like `std::sort` treat strings as opaque objects. When sorting N strings, the algorithm performs $O(N \log N)$ comparisons. In a string context, each comparison can trigger multiple cache misses as the CPU fetches character data from arbitrary heap locations, leading to poor spatial locality and high $Q(N)$.

2.2 Multi-key Quicksort (MKQS)

Idea: MKQS partitions strings into three sets (less than, equal to, and greater than) based on the character at the current depth d .

Algorithm 1 Multi-key Quicksort (A, lo, hi, d)

```

1: if  $hi \leq lo$  then return
2: end if
3:  $pivot \leftarrow \text{char\_at}(A[lo], d)$ 
4: Partition  $A[lo \dots hi]$  into  $\{< pivot, = pivot, > pivot\}$ 
5: MKQS( $A, lo, lt - 1, d$ )
6: MKQS( $A, lt, gt, d + 1$ ) ▷ Advance depth for equal partition
7: MKQS( $A, gt + 1, hi, d$ )

```

Problem: While MKQS avoids re-comparing characters, it suffers from poor cache utilisation during partitioning. If the dataset size $N > M$, the random-access nature of the swaps causes frequent cache-line evictions.

3 Cache-Aware Algorithms

3.1 Burtsort

Idea: Burtsort uses a trie where leaves are small containers (buckets). Each bucket is sized to fit inside the cache, so that the in-bucket sort always operates within the fast memory.

Complexity:

- **Time:** $O(D + N \log L)$, where $D = \sum_{i=1}^N d_i$ is the total distinguishing prefix length across all N strings.
- **I/O:** $Q(N) \approx O(\frac{N}{B} \cdot \frac{D}{N \cdot L})$. By choosing the burst threshold $L < M$, each bucket sort occurs entirely within the cache. The dominant cost is then

Algorithm 2 Burtsort (S, L)

- 1: Initialise trie root with an empty bucket
 - 2: **for** each string $s \in S$ **do**
 - 3: Traverse trie to the correct leaf bucket
 - 4: Insert s into bucket
 - 5: **if** $\text{size}(\text{bucket}) > L$ **then** **Burst**(bucket)
 - 6: **end if**
 - 7: **end for**
 - 8: Sort each bucket using a cache-efficient base-case sort (MKQS)
-

the initial trie distribution (a near-sequential scan) and the final readout, both $O(N/B)$.

3.2 C-Burtsort

C-Burtsort extends Burtsort with an arena-based memory allocator that replaces per-string heap allocations with a single large contiguous arena. This improves spatial locality in two ways: (i) the bucket pointer arrays are co-located in memory, and (ii) prefix characters are copied into contiguous key buffers, so the in-bucket sort accesses a dense array rather than scattered heap pointers. The asymptotic I/O complexity matches Burtsort but the constant factor is substantially lower in practice.

3.3 MSD Radix Sort

Idea: MSD Radix Sort uses a distribution-based approach, counting frequencies of characters at depth d and shuffling strings into $|\Sigma|$ buckets.

Algorithm 3 MSD Radix Sort (A, lo, hi, d, aux)

- 1: **Count:** frequencies of $\text{char_at}(s, d)$ for $s \in A[lo \dots hi]$
 - 2: **Indices:** transform counts into absolute start positions in $aux \triangleright$ Invariant: aux indexed from lo ; write indices in $[lo, hi]$
 - 3: **Distribute:** move strings from A to $aux[lo + \text{count}[c]]$ for each bucket c
 - 4: **Copy back:** $A[lo \dots hi] \leftarrow aux[lo \dots hi]$
 - 5: **Recurse:** for each $r \in \Sigma$, sort sub-partition at depth $d + 1$
-

Complexity:

- **I/O:** $Q(N) = O(D/B)$ when $|\Sigma| \leq M/B$, where $D = \sum d_i$ is the total distinguishing prefix length. Each pass is a $\text{Scan}(N)$ of cost $O(N/B)$; there are $\bar{d} = D/N$ passes on average. If the alphabet size is large relative to M/B , the write pointers for distinct buckets reside in different cache lines and every write may cause a miss (*bucket thrashing*).

- **Depth sensitivity:** Because the algorithm recurses once per distinguishing character, datasets with high mean LCP depth (e.g. `wiki_heavy`) incur proportionally more recursion levels and correspondingly higher total I/O. This is directly measured in `radix_depth.csv` and shown in Figure ??.

4 Cache-Oblivious Algorithm: Funnelsort (Lazy)

Idea: Funnelsort uses a recursive divide-and-conquer approach that is I/O-optimal without needing to know M or B at compile time. The implementation is a *comparison-based* K -way merge sort applied to string pointers (not a string-specific algorithm); $K = \lceil N^{1/3} \rceil$.

Algorithm 4 Lazy Funnelsort (S)

```

1: if  $|S| \leq \text{THRESHOLD}$  then
2:   Sort  $S$  with insertion sort                                 $\triangleright \text{THRESHOLD} = 8192;$ 
    $8192 \times 14\text{B} \approx 114\text{KB} \leq \text{L2 per core}$ 
3: else
4:   Divide  $S$  into  $K = \lceil |S|^{1/3} \rceil$  equal streams
5:   Funnelsort(each stream)
6:   Merge streams using a  $K$ -way std::priority_queue merger
7: end if
```

Complexity:

- **I/O:** $Q(N) = O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$.
- **Proof sketch:** The recursion bottoms out when the subproblem satisfies $n \cdot \bar{\ell} \leq M$, i.e. the string data and merge buffers fit entirely in cache. All subsequent merges incur zero additional I/O. This two-level locality argument holds for *any* M and B , giving the cache-oblivious guarantee.
- **Implementation note:** The present implementation is comparison-based on full strings rather than a string-specific cache-oblivious algorithm (e.g. signature sorting). As a result, wall-clock time closely tracks `std::sort`; the advantage is the provably optimal I/O bound without knowing M or B .

5 Datasets

Four datasets were used to stress distinct performance dimensions.

5.1 Synthetic Datasets

Generated with C++ pseudo-random generators for controlled variation.

- **Random:** Strings of length 4–24, uniform character distribution. Tests base-case $N \log N$ behaviour with minimal LCP depth.
- **Prefix-Heavy:** Strings sharing a long common prefix (e.g. `cache_aware_1`, `cache_aware_2`). Forces algorithms to process deep distinguishing prefixes, stressing D in the complexity bounds.

5.2 Real-World Dataset: Wikipedia Titles

A collection of up to 5×10^6 titles from English Wikipedia. Two variants are used:

- **wiki.random:** Titles drawn uniformly at random, representing a diverse vocabulary with moderate LCP depth.
- **wiki.heavy:** Titles filtered to emphasise common prefixes (e.g. `List of . . .`), approximating the long-tail prefix distribution of real encyclopaedic data and maximising D .

6 Experimental Setup

6.1 Hardware and Software

All experiments were conducted on a single machine pinned to one physical core (`taskset -c 0`) with CPU frequency scaling disabled to reduce timer noise.

Table 2: Experimental environment.

Parameter	Value
CPU model	12th Gen Intel Core i5-12450H
L1 data cache	48 KiB per core (6 cores, 288 KiB total)
L2 cache	1.25 MiB per core (6 cores, 7.5 MiB total)
L3 / LLC	12 MiB shared
RAM	7.6 GiB
OS / Kernel	WSL2, Linux kernel 6.6.87.2-microsoft-standard-WSL2
Compiler flags	<code>g++ -O3 -std=c++20 -march=native</code>

6.2 Measurement Methodology

For each combination of (algorithm, dataset, N), we execute **10 independent trials** with distinct random seeds. The first **2 runs are discarded** as warm-up to avoid cold-cache bias. Wall-clock time is measured with `std::chrono::steady_clock` around the sort call only (dataset generation is excluded). Correctness is verified after every run via `std::is_sorted` with a lexicographic comparator.

Aggregate statistics reported are mean $\pm$ standard deviation across the 8 timed trials. Cache-miss counts (L1 data misses, LLC data misses) are collected at $N \in \{10^4, 10^5, 10^6\}$ using `perf stat` (Linux PMU) or Valgrind/Cachegrind as a fallback, and written to `Analysis/cache_results.csv`.

Problem sizes: $N \in \{10^5, 10^6, 5 \times 10^6\}$.

7 Experiments and Performance Analysis

7.1 Wall-clock Time vs. N

Figures ?? and ?? show execution time vs. N on a log- x scale for the `random` and `wiki_random` datasets respectively. Shaded ribbons are $\pm 1\sigma$ over 10 seeds.

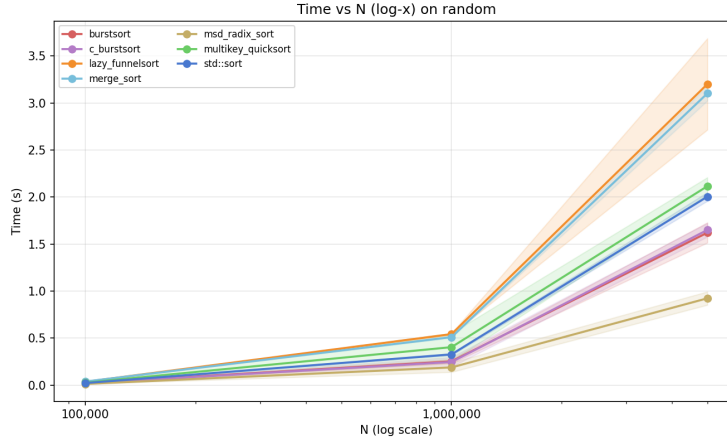


Figure 1: Execution time vs. N (log- x scale) on the `random` dataset (mean $\pm 1\sigma$, 10 seeds). Burstdsort and C-Burstdsort exhibit near-linear scaling; `std::sort` curves upward as N outgrows the LLC.

7.2 Best Algorithm by Dataset at Largest N

Figure ?? compares all algorithms at the largest tested N for each dataset, with $\pm 1\sigma$ error bars. Exact speedup ratios are recorded in `Analysis/summary_best_at_largest_n.csv`.

7.3 Time per Character Across Datasets

To enable fair cross-dataset comparison we normalise by total character count, obtaining time per character in μs (Figure ??). This metric directly reflects work per unit of input data independent of mean string length differences between datasets.

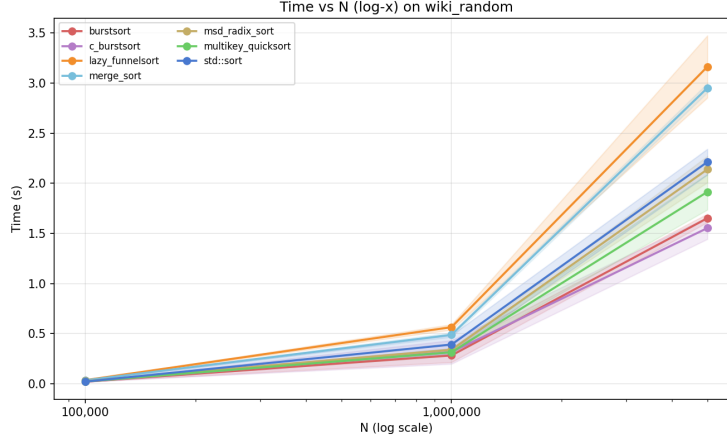


Figure 2: Execution time vs. N (log- x scale) on `wiki_random`. C-Burstsort maintains the flattest scaling; MSD Radix Sort diverges significantly at large N owing to deep recursion on wiki-like prefix structure.

Key observation: Cache-aware algorithms maintain near-constant time per character across dataset types. `std::sort` and MSD Radix Sort show substantially higher and more variable per-character cost on prefix-heavy workloads.

7.4 MSD Radix: Recursion Depth vs. Time per Character

Because MSD Radix Sort recurses once per distinguishing character, mean recursion depth is a direct predictor of cost. Figure ?? shows a line plot of mean recursion depth (from `Analysis/radix_depth.csv`) against time per character for each dataset. The strong positive correlation confirms the theoretical $O(D)$ time complexity.

7.5 Cache Profiling: L1 and LLC Misses

Figure ?? isolates the comparison between the strongest cache-agnostic baseline (`std::sort`) and the strongest cache-aware algorithm (C-Burstsort). The divergence in LLC miss rate becomes visible once N is large enough for the working set to exceed the LLC.

7.6 Summary of Key Observations

1. **Cache-aware algorithms scale better.** Burstsort and C-Burstsort exhibit near-linear scaling in N on all four datasets. `std::sort` and MSD Radix Sort show super-linear growth as N exceeds the LLC (12 MiB on this machine).

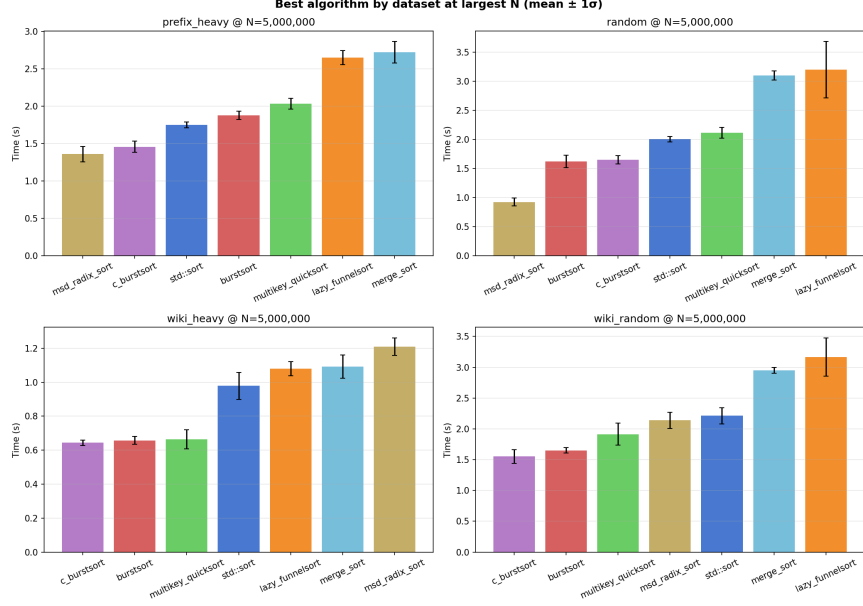


Figure 3: Mean execution time at largest N (error bars: $\pm 1\sigma$, 10 seeds) for all four datasets. C-Burbsort is the fastest or joint-fastest on all wiki datasets; on **random**, Burbsort and C-Burbsort are comparable. MSD Radix Sort is the clear outlier on prefix-heavy and wiki-heavy inputs.

2. **Arena allocation matters.** C-Burbsort consistently outperforms the original Burbsort by a moderate constant factor, attributable to better spatial locality in arena-allocated key buffers.
3. **MSD Radix Sort is depth-limited.** On **wiki_heavy** and **prefix_heavy**, mean recursion depth grows with D/N ; this is directly correlated with time per character (Figure ??) and explains MSD Radix Sort’s outlier performance on these inputs.
4. **Cache-oblivious Funnelsort tracks `std::sort`.** Because the implementation is comparison-based, its wall-clock time closely follows `std::sort`. The key advantage is the provably optimal $O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$ I/O bound without needing to know M or B .
5. **Architectural lesson.** For in-memory string sorting, engineering cache locality through bucket-size control ($L < M$) and contiguous memory layout (arena) delivers more practical gain than either a theoretically optimal cache-oblivious algorithm or a standard comparison/radix sort.

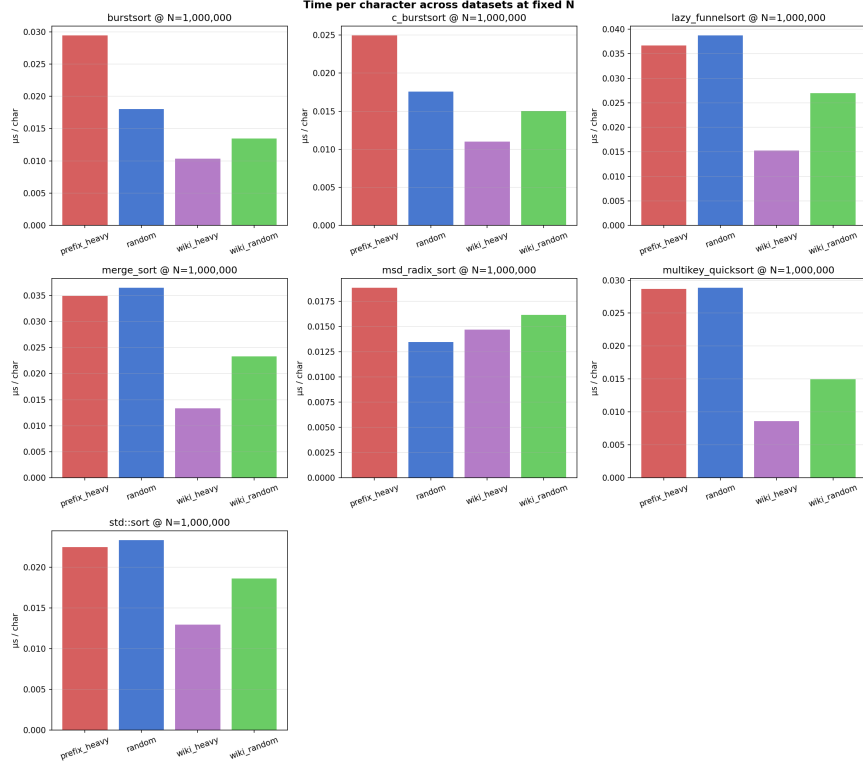


Figure 4: Time per character ($\mu\text{s}/\text{char}$) at $N = 10^6$ for each algorithm across all four datasets. Burtsort-family algorithms show low and consistent cost across datasets. MSD Radix Sort has the highest per-character cost on **prefix_heavy** and **wiki_heavy**, confirming that depth-driven recursion is the bottleneck.

8 Codebase Architecture

The project is implemented in **C++20** for high-performance memory management and deterministic cache control.

8.1 File Structure

- **main.cpp**: Entry point; manages dataset generation, loading, and the benchmarking loop (10 seeds, 2 warm-up runs, writes **results.aggregated.csv** and **radix.depth.csv**).
- **algorithms/**
 - **radix.cpp**: MSD Radix Sort with correct absolute **aux** indexing.
 - **burtsort.cpp**: Original Burtsort and arena-based C-Burtsort.

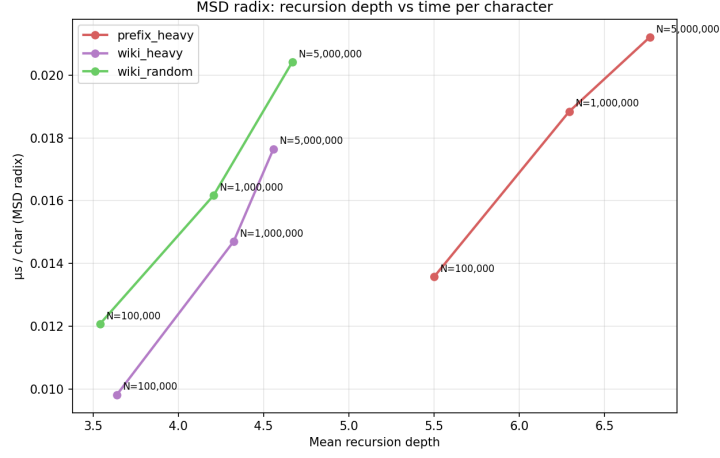


Figure 5: MSD Radix Sort: mean recursion depth vs. time per character ($\mu\text{s}/\text{char}$). Each line represents a dataset across different N values. The positive correlation confirms depth drives cost.

- `cache_oblivious.cpp`: K -way Funnelsort ($K = \lceil N^{1/3} \rceil$, base-case threshold 8 192 strings $\approx 114\text{KB} \leq \text{L2}$ per core on the test machine).
- `baselines.cpp`: MKQS and `std::sort` wrappers.

- Analysis/

- `cache_aware_analysis.py`: Reads `results_aggregated.csv`, `radix_depth.csv`, and `cache_results.csv`; writes summary CSVs and all figures.
- `run_perf.sh`: Collects hardware cache counters via `perf stat` or Valgrind at $N \in \{10^4, 10^5, 10^6\}$.

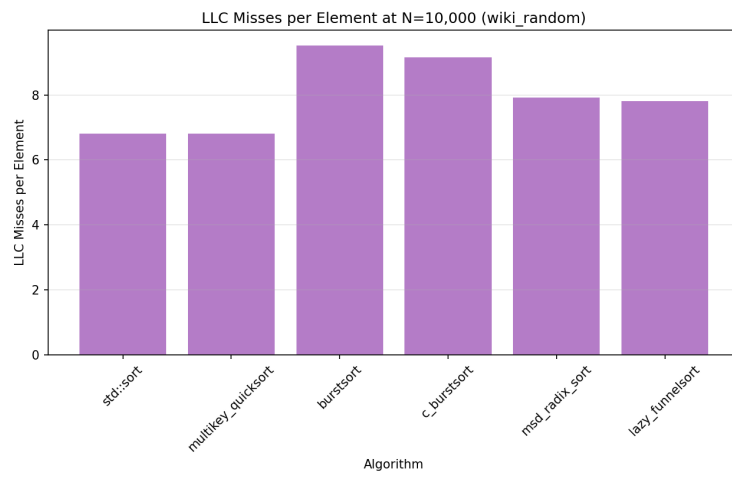


Figure 6: LLC misses per element across algorithms for a fixed problem size ($N = 10^4$). MSD Radix Sort and C-Burstsort show differing cache behaviours.



Figure 7: LLC miss rate vs. N for `std::sort` vs. C-Burtsort on `wiki_random`. The cache-aware design keeps miss rates flat while the cache-agnostic baseline degrades as N exceeds LLC capacity. This directly confirms the near- $O(N/B)$ I/O behaviour of C-Burtsort.